

React + Vite Setup Cheat Sheet

Everything to go from empty folder to running React + TypeScript app · Vite 8 era

1 Check Node first

once per machine

```
node -v # need 20.19+ or 22.12+
npm -v
```

Too old? Download the LTS build from nodejs.org and reopen your terminal.

2 Create the project

```
npm create vite@latest my-app -- --template react-ts
cd my-app
npm install
npm run dev
```

The `--` is required with npm. Without it, npm eats the `--template` flag.

TEMPLATES

react-ts	React + TypeScript ← use this
react	React + JavaScript
react-swc-ts	Same as react-ts, faster compiler

Or run `npm create vite@latest` alone and answer the prompts.

3 Which linter did you get?

Look in your project root. You have exactly one of these — **already configured**. Do not install a linter.

► .oxlintrc.json → Oxlint

VS Code ext	Oxlint · <code>oxc.oxc-vscode</code>
Extra setup	None. Skip the Prettier conflict fix.

► eslint.config.js → ESLint

VS Code ext	ESLint · <code>dbaeumer.vscode-eslint</code>
Extra setup	Add eslint-config-prettier (step 4)

Both do the same job: underline mistakes as you type. Run either with `npm run lint`.

4 Add Prettier

not included

```
npm install --save-dev prettier
```

ONLY IF YOU HAVE ESLINT

```
npm install --save-dev eslint-config-prettier
```

Then in `eslint.config.js`, add as the **last** array item:

```
import eslintConfigPrettier from 'eslint-config-prettier'
// ...
export default [ /* ...configs */, eslintConfigPrettier ]
```

CREATE .PRETTIERRC IN THE ROOT

```
{
  "singleQuote": true,
  "arrowParens": "avoid"
}
```

ADD TO PACKAGE.JSON → "SCRIPTS"

```
"format": "prettier --write ."
```

5 VS Code extensions

EXTENSION	ID
Prettier everyone	<code>esbenp.prettier-vscode</code>
Oxc if Oxlint	<code>oxc.oxc-vscode</code>
ESLint if ESLint	<code>dbaeumer.vscode-eslint</code>
Error Lens optional	<code>usernamehw.errorlens</code>
ES7+ Snippets optional	<code>dsznajder.es7-react-js-snippets</code>

Restart VS Code after installing.

6 Turn on format-on-save

`Ctrl+Shift+P` → "Preferences: Open User Settings (JSON)" → add inside the braces:

```
"editor.formatOnSave": true,
"editor.defaultFormatter": "esbenp.prettier-vscode"
```

✓ **Setup done.** Steps 1–6 are one-time per project (1, 5, 6 are one-time per machine).

Daily commands

<code>npm run dev</code>	Start dev server → localhost:5173
<code>npm run build</code>	Type-check + build to <code>dist/</code>
<code>npm run preview</code>	Preview the built app
<code>npm run lint</code>	Run Oxlint / ESLint
<code>npm run format</code>	Prettier every file
<code>npm install</code>	Restore <code>node_modules</code>
<code>Ctrl + C</code>	Stop the dev server

DEV SERVER FLAGS

```
npm run dev -- --port 3000
npm run dev -- --open # auto-open browser
npm run dev -- --host # test on your phone
```

TypeScript: the one gotcha

Vite does not type-check while `npm run dev` is running. It strips types and moves on — that is why it is fast. A broken type will still hot-reload happily.

TWO SAFETY NETS

- **Your editor.** VS Code underlines type errors live. Catches ~95%.
- **A watcher** in a second terminal tab:

```
npx tsc --noEmit --watch
```

`npm run build` does type-check (`tsc -b && vite build`), so nothing broken ever ships.

Project map

my-app/
├ index.html ← entry HTML (root!)
├ package.json scripts + deps
├ vite.config.ts plugins
├ tsconfig.json TS rules
├ .prettierrc you made this
├ .oxlintrc.json or eslint.config.js
├ public/ static, copied as-is
└ src/
 ├ main.tsx ← app entry
 ├ App.tsx root component
 ├ index.css global styles
 └ assets/ imported files

Never edit node_modules/ or dist/. Both are rebuildable and git-ignored.

Environment variables

Create `.env` in the project root:

VITE_API_KEY=abc123

Use it in code:

import.meta.env.VITE_API_KEY

Must start with `VITE_` or it is invisible. Values ship to the browser — **never put real secrets here**. Restart the dev server after editing `.env`.

Coming from create-react-app

CRA	VITE
npm start	npm run dev
src/index.js	src/main.tsx
public/index.html	index.html (root)
process.env .REACT_APP_X	import.meta.env .VITE_X
Jest	Vitest
localhost:3000	localhost:5173
webpack	Rolldown / esbuild

CRA was formally deprecated by the React team in Feb 2025 and gets no security patches. Use it only to follow along with older tutorials.

Common add-ons

ROUTING

npm i react-router-dom

TAILWIND CSS V4

npm i tailwindcss @tailwindcss/vite

vite.config.ts:

import tailwindcss from '@tailwindcss/vite'
plugins: [react(), tailwindcss()]

src/index.css — replace everything with:

@import "tailwindcss";

TESTING

npm i -D vitest jsdom @testing-library/react @testing-library/jest-dom

DATA FETCHING / STYLING

npm i @tanstack/react-query
npm i styled-components

Troubleshooting

SYMPTOM	FIX
"Need to install the following packages: tsc"	Answer n . You are in a folder with no TypeScript. <code>cd</code> into your Vite project first.
Port 5173 already in use	<code>npm run dev -- --port 3000</code> , or close the other terminal.
"Cannot find module" after cloning or pulling	Delete <code>node_modules</code> + <code>package-lock.json</code> , then <code>npm install</code> .
Prettier not formatting on save	Status bar (bottom-right) should read "Prettier". If not: <code>Ctrl+Shift+P</code> → "Format Document With..." → set default.
Changed vite.config.ts, nothing happened	Restart the dev server. Config is not hot-reloaded.
Type error invisible in browser	Expected. See the TypeScript card.
Blank page after deploying	Set <code>base: '/repo-name/'</code> in <code>vite.config.ts</code> if hosting in a subfolder.

The npx rule

`npx` runs a tool from your project — and **silently downloads a stranger's package from the internet** if it is not there.

If npx ever asks "**Ok to proceed? (y)**" for a tool you expected to already have — press **n**. You are almost always in the wrong folder.

Git starter

git init
git add .
git commit -m "initial commit"

Vite generates a `.gitignore` that already excludes `node_modules/`, `dist/` and `.env`. Nothing to configure.